GESP-7 质量抽检模拟试卷

考试时间：60 分钟　满分：100 分　生成时间：2026/05/07 08:39:56

姓名：_____ 班级：_____ 得分：_____

请在答题区域作答；单项选择题、阅读程序题和完善程序题均为单选。除特别说明外，程序均按 C++17 标准理解。

抽样来源：papers/2026 已入库题库文件仓库；蓝图版本：config/blueprint.ts v1；抽样种子：20260507034045

一、单项选择题（共 15 题，每题 2 分，共 30 分）

1. （2 分）GESP-7 算法单选：使用深度优先搜索（DFS）枚举数组 [2, 3, 5, 7, 8] 的所有子集，统计其中元素之和恰好为 15 的子集个数。结果是多少？

A. 4
B. 5
C. 2
D. 3

2. （2 分）给定数组 arr = {8, 3, 5, 2, 9, 1, 7}，对 arr 执行一趟快速排序（以第一个元素为基准，使用 Lomuto 分区方案），请问一趟分区后，基准元素 8 的最终索引是多少？

A. 2
B. 3
C. 4
D. 5

3. （2 分）GESP-7 算法单选 147：DFS 枚举数组 [2, 3, 5, 7, 8] 的子集，和恰好为 15 的子集个数是多少？（案例线索：median-orbit-restore-147，关注 binary-answer feasibility。）

A. 4
B. 5
C. 2
D. 3

4. （2 分）GESP-7 算法单选 332：给定集合 S = {2, 3, 5, 4, 8}，使用深度优先搜索（DFS）枚举所有子集（包括空集）。请问有多少个子集的元素之和恰好等于 15？

A. 2
B. 3
C. 4
D. 1

5. （2 分）GESP-7 算法单选 340：有序数组 [6, 9, 12, 15, 18, 21, 24] 中用 lower_bound 查找 24，返回位置按 1 开始计数，结果是多少？要求：解释 lower_bound 的二分查找过程，并说明当目标值存在时的行为。（案例线索：二分查找边界，关注返回位置定义。）

A. 7
B. 6
C. 8
D. 5

6. （2 分）GESP-7 数据结构单选 721：使用队列模拟约瑟夫环问题。有7个人，编号从1开始依次为1,2,3,4,5,6,7。从编号1开始报数，每次报数到3的人出队，直到剩下最后一人。请问最后留下的人的编号是几？

A. 4
B. 5
C. 6

D. 7

7. （2 分）对关键字序列 (15, 9, 20, 12, 17, 11) 建立小根堆，堆调整后，堆顶元素是？

A. 9

B. 11

C. 12

D. 15

8. （2 分）对于长度为n的顺序表，在表头插入一个元素的时间复杂度是？

A. O(1)

B. O(log n)

C. O(n)

D. O(n^2)

9. （2 分）对于如下 C++ 代码，程序输出的结果是？

```
#include <iostream>
#include <stack>
using namespace std;
int main() {
    stack<int> s;
    int a[] = {1, 3, 2, 5, 4};
    for (int i = 0; i < 5; i++) {
        if (s.empty() || a[i] < s.top()) {
            s.push(a[i]);
        } else {
            while (!s.empty() && a[i] > s.top()) {
                s.pop();
            }
            s.push(a[i]);
        }
    }
    cout << s.size();
    return 0;
}
```

A. 2

B. 3

C. 4

D. 5

10. （2 分）GESP-7 数据结构单选 209：给定字符串序列 ['aa', 'bb', 'aa', 'c1', 'bb', 'dd']，使用集合（set）统计其中不同字符串的个数，结果是多少？

A. 3

B. 4

C. 5

D. 6

11. （2 分）GESP-7 C++ 单选 1106：给定一个初始为空的 vector<int> v，依次执行以下操作：1. v.push_back(11); 2. v.push_back(7); 3. v.insert(v.begin(), 3); 4. v.erase(v.begin() + 1); 5. v.pop_back(); 请问最终 v 中的元素序列是什么？

A. 3

B. 7

C. 3,11

D. 11,3

12. （2 分）GESP-7 C++ 单选 498：执行以下代码：

```
int a = 4;
void inc(int& x) { x++; }
```

inc(a);
cout << a + 6;
输出结果是多少?

A. 10
B. 7
C. 11
D. 5

13. （2 分）GESP-7 C++ 单选 814：外层 int x=8; 内层代码块声明 int x=18; 离开内层后输出 x，结果是多少?

A. 18
B. 26
C. 8
D. 10

14. （2 分）GESP-7 数学单选 811：埃氏筛统计不超过 41 的质数个数，结果是多少？（案例线索：forest-packet-count-811，关注 prime-sieve crossing。）

A. 12
B. 14
C. 20
D. 13

15. （2 分）GESP-7 数学单选 1133：计算 10^6 mod 17 的结果是多少?

A. 10
B. 9
C. 12
D. 60

二、阅读程序题（共 3 题，每题 5 小题，共 40 分；小题分值见题内标注）

16. （本题 14 分）GESP-7 程序阅读 885（ALG/medium）：阅读下面的 C++17 程序并回答问题。

```
// case prefix-vertex-filter-885; focus binary-answer feasibility; level medium; kp ALG
// OJCode-inspired training note: prefix-vertex-filter-885
#include <bits/stdc++.h>
using namespace std;

int main() {
    vector<int> a = {3, 5, 7, 9, 11, 13, 15, 17};
    int target;
    cin >> target;
    int l = 0, r = (int)a.size() - 1;
    int ans = -1;
    while (l <= r) {
        int mid = l + (r - l) / 2;
        if (a[mid] >= target) {
            ans = mid;
            r = mid - 1;
        } else {
            l = mid + 1;
        }
    }
    cout << ans << '\n';
    return 0;
}
```

16.1 （3 分）当 target 取 3 时，程序输出的下标是多少？（观察点 1：prefix-vertex-filter-885）

A. 1

B. 0

C. -1

D. 0 的干扰项 1

16.2 （3 分）当 a[mid] >= target 时，程序下一步如何缩小区间？（观察点 2：prefix-vertex-filter-885）

A. l=mid+1

B. r=mid-1

C. 直接退出

D. 删除 a[mid]

16.3 （3 分）ans 的含义是什么？（观察点 3：prefix-vertex-filter-885）

A. 当前候选的第一个不小于 target 的位置

B. target 的出现次数

C. 数组长度

D. 当前最大值

16.4 （3 分）这段查找的时间复杂度是多少？（观察点 4：prefix-vertex-filter-885）

A. O(1)

B. O(log n)

C. O(n)

D. O(n^2)

16.5 （2 分）数组有序是该算法正确的必要条件吗？（观察点 5：prefix-vertex-filter-885）

A. 是

B. 否

C. 只对奇数长度是

D. 只对 target 为偶数是

17. （本题 13 分）给定以下C++17程序，当输入为5时，程序会执行一些计算。请根据程序逻辑回答问题。

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; ++i) {
        a[i] = i * i - 4 * i + 5;
    }
    int max_val = a[0];
    int min_val = a[0];
    for (int i = 1; i < n; ++i) {
        if (a[i] > max_val) max_val = a[i];
        if (a[i] < min_val) min_val = a[i];
    }
    int diff = max_val - min_val;
    vector<int> b;
    for (int i = 0; i < n; ++i) {
        if (a[i] == max_val || a[i] == min_val) {
            b.push_back(i);
        }
    }
    cout << diff << endl;
    cout << b.size() << endl;
    for (int i = 0; i < b.size(); ++i) {
        cout << b[i] << (i == b.size() - 1 ? "" : " ");
    }
```

```
    return 0;
}
```

```
4
3
0 2 4
```

17.1 （3 分）程序的第一行输出（即diff的值）是多少?

A. 4

B. 8

C. 12

D. 16

17.2 （3 分）程序的第二行输出（即b的大小）是多少?

A. 1

B. 2

C. 3

D. 4

17.3 （3 分）程序的第三行输出（即b的元素，以空格分隔）是什么?

A. 0 2 4

B. 0 1 4

C. 1 3 4

D. 0 3 4

17.4 （2 分）在程序执行过程中，变量max_val被更新的次数是多少?

A. 0

B. 1

C. 2

D. 3

17.5 （2 分）如果输入n=5，程序中数组a的第4个元素（即a[3]）的值是多少?

A. 1

B. 2

C. 3

D. 4

18. （本题 13 分）阅读以下C++程序，根据给定的输入，回答后续问题。

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    int n = 6;
    int arr[] = {3, 1, 4, 2, 5, 1};
    stack<int> s;
    int result = 0;
    for (int i = 0; i < n; i++) {
        while (!s.empty() && s.top() <= arr[i]) {
            s.pop();
        }
        s.push(arr[i]);
        result += s.size();
    }
    cout << "最终栈大小: " << s.size() << endl;
    cout << "栈顶元素: " << s.top() << endl;
    cout << "累积和result: " << result << endl;
    return 0;
```

```
}
```

```
最终栈大小: 2
栈顶元素: 1
累积和result: 9
```

18.1 （3 分）程序最终输出的“最终栈大小”是多少？

A. 1

B. 2

C. 3

D. 4

18.2 （3 分）程序最终输出的“栈顶元素”是多少？

A. 1

B. 2

C. 5

D. 9

18.3 （3 分）程序最终输出的“累积和result”是多少？

A. 7

B. 8

C. 9

D. 10

18.4 （2 分）在循环执行过程中，栈` s `被弹出（pop）操作的总次数是多少？

A. 3

B. 4

C. 5

D. 6

18.5 （2 分）循环执行过程中，栈`s`的大小（元素个数）的最大值是多少？

A. 1

B. 2

C. 3

D. 4

三、完善程序题（共 2 题，共 30 分；各空分值见题内标注）

19. （本题 15 分，共 4 空）下面程序实现了简单选择排序。请填入正确代码完成排序功能。

```
#include <iostream>
#include <vector>
using namespace std;

void selectionSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < BLANK1; ++i) {
        int minIndex = i;
        for (int j = BLANK2; j < n; ++j) {
            if (BLANK3) minIndex = j;
        }
        if (minIndex != i) BLANK4;
    }
}

int main() {
    int n;
    cin >> n;
    vector<int> arr(n);
    for (int i = 0; i < n; ++i) cin >> arr[i];
```

```
    selectionSort(arr);
    for (int i = 0; i < n; ++i) cout << arr[i] << " ";
    cout << endl;
    return 0;
}
```

样例输入：

```
5
3 1 4 1 5
```

样例输出：

```
1 1 3 4 5
```

空 1（BLANK1，4 分）

A. n - 1

B. n

C. n / 2

D. n + 1

空 2（BLANK2，4 分）

A. i + 1

B. i

C. 0

D. i - 1

空 3（BLANK3，4 分）

A. arr[j] < arr[minIndex]

B. arr[j] > arr[minIndex]

C. arr[j] == arr[minIndex]

D. arr[j] != arr[minIndex]

空 4（BLANK4，3 分）

A. swap(arr[i], arr[minIndex])

B. swap(arr[minIndex], arr[i])

C. arr[i] = arr[minIndex]

D. arr[minIndex] = arr[i]

20. （本题 15 分，共 4 空）以下C++代码实现了一个简易的基于数组的环形队列，其中`front`和`rear`分别指向队首和队尾的下一个位置。队列的最大容量为`capacity`，当前元素个数由`size`变量维护。请补全代码中的空白，以实现入队和出队操作。

```
#include <iostream>
using namespace std;

const int MAXN = 100;

class CircularQueue {
private:
    int data[MAXN];
    int front, rear, size, capacity;
public:
    CircularQueue(int cap) {
        front = 0;
        rear = 0;
        size = 0;
        capacity = cap;
    }

    bool enqueue(int x) {
        if (BLANK1) {
            return false; // 队列已满
        }
        data[rear] = x;
        BLANK2;
```

```
        size++;
        return true;
    }

    bool dequeue(int &x) {
        if (BLANK3) {
            return false; // 队列为空
        }
        x = data[front];
        BLANK4;
        size--;
        return true;
    }
};

int main() {
    CircularQueue q(5);
    int val;
    q.enqueue(10);
    q.enqueue(20);
    if (q.dequeue(val)) {
        // 首次出队，val被赋值
    }
    q.enqueue(30);
    q.enqueue(40);
    q.enqueue(50);
    q.enqueue(60); // 此时应满
    if (q.dequeue(val)) {
        // 第二次出队，val被赋值
    }
    q.enqueue(70);
    while (q.dequeue(val)) {
        cout << val << " ";
    }
    return 0;
}
```

样例输入：

样例输出：

```
30 40 50 60 70
```

空 1（BLANK1，4 分）

A. size == capacity

B. front == rear

C. size == MAXN

D. rear == capacity

空 2（BLANK2，4 分）

A. rear = (rear + 1) % capacity

B. rear = (rear + 1) % MAXN

C. rear++

D. rear = rear % capacity + 1

空 3（BLANK3，4 分）

A. front == rear

B. size == 0

C. front == 0 && rear == 0

D. size != capacity

空 4（BLANK4，3 分）

A. front = (front + 1) % capacity

B. front = (front + 1) % MAXN

C. front++

D. front = (front + 1) % size